Architektur und Optimierung eines Ultra-Low-Latency Solana-Trading-Nodes auf Hetzner AX42-U Bare Metal

Einleitung und Systemarchitektur

In den hochkompetitiven und asynchronen Märkten des Decentralized Finance (DeFi) Ökosystems der Solana-Blockchain erfordert die erfolgreiche Implementierung prädiktiver Hochfrequenzhandelsstrategien (HFT) eine kompromisslose Minimierung der Latenz auf allen Ebenen der Systemarchitektur. Das spezifische Ziel dieser Analyse ist die architektonische und systemnahe Optimierung für die Vorhersage und Ausführung von Short-Positionen bei rapiden, asymmetrischen Preisbewegungen von über drei Prozent (>3% SHORT). Derartige Marktereignisse – oft ausgelöst durch kaskadierende Liquidierungen in Lending-Protokollen, massive Swap-Transaktionen in Automated Market Makers (AMMs) wie Raydium oder Orca, oder abrupte Aktualisierungen von On-Chain-Orakeln (z.B. Pyth Network) – erfordern eine Datenverarbeitung in Echtzeit, die weit unterhalb der Latenz gewöhnlicher RPC-Knoten operiert.

Um diese Marktanomalien zu antizipieren, bevor sie im finalisierten Ledger konsolidiert werden, müssen rohe Netzwerkdaten (Shreds) direkt von den Leader-Nodes ingestiert, dekodiert und durch komplexe algorithmische Modelle bewertet werden. Die physische und logische Basis für diese Infrastruktur bildet der dedizierte Bare-Metal-Server AX42-U von Hetzner Online. Diese Plattform bietet eine außergewöhnlich kosteneffiziente, aber gleichzeitig rechenstarke Grundlage.[1] Ausgestattet mit einem AMD Ryzen 7 PRO 8700GE Prozessor, der auf der Zen-4-Architektur ("Phoenix") basiert, 64 Gigabyte DDR5 ECC-Arbeitsspeicher und zwei 512 Gigabyte Gen4 NVMe Solid-State-Drives im Software-RAID 1, liefert der Server die notwendige Hardware-Ressource für hochgradig parallelisierte und speicherintensive Krypto-Workloads.[2]

Trotz der Limitierung der Netzwerkschnittstelle auf einen 1 Gigabit pro Sekunde (Gbit/s) Port [3] bietet die physische Lokalisierung der Hetzner-Rechenzentren (insbesondere Falkenstein und Nürnberg) durch direkte Glasfaseranbindungen an den DE-CIX in Frankfurt am Main einen unschätzbaren topologischen Vorteil.[4] Diese Nähe zu kritischer Solana-Infrastruktur, explizit der Jito Labs Block Engine, bildet das Fundament der Netzwerkoptimierung.[6]

Dieser Forschungsbericht detailliert die tiefgreifende Optimierung des gesamten Hardware- und Software-Stacks. Um die für prädiktive >3% Short-Modelle essenziellen Mikrosekunden-Reaktionszeiten zu gewährleisten, muss jegliches nicht-deterministische Verhalten, jeglicher Jitter des Betriebssystems (OS) und jeder Overhead durch Kontextwechsel (Context Switching) eliminiert werden. Dies umfasst die topologische Abstimmung des AMD

L3-Caches, die vollständige Umgehung des Linux-Kernel-Netzwerkstacks mittels AF_XDP (Address Family eXpress Data Path), die Eliminierung der zustandsbehafteten Firewall-Komponente nf_conntrack, direkte Hardware-Zeitstempel via Precision Time Protocol (PTP), sowie die asynchrone Datenpersistierung über DuckDB unter Nutzung von io_uring und O_DIRECT. Die Datenaufnahme stützt sich dabei auf Jito Shredstream zur Umgehung des Turbine-Propagationsbaums, gepaart mit latenzoptimierten Yellowstone gRPC-Streams für die Verifikation von Account-Zuständen.

Hardware-Topologie und strikte CPU-Isolierung: AMD Ryzen 7 PRO 8700GE

Das rechnerische Herzstück des Hetzner AX42-U ist der AMD Ryzen 7 PRO 8700GE Prozessor. Diese CPU verfügt über acht physische Kerne und sechzehn logische Threads, taktet mit einem Basistakt von 3,6 GHz und erreicht Boost-Frequenzen von bis zu 5,1 GHz.[7] Der Prozessor wird im 4-Nanometer-FinFET-Verfahren von TSMC gefertigt und zeichnet sich durch eine Default Thermal Design Power (TDP) von lediglich 35 Watt aus, was ihn zu einer äußerst thermisch stabilen und energieeffizienten Recheneinheit macht.[7]

Die monolithische L3-Cache-Architektur

Im Gegensatz zu den leistungsstärkeren AMD EPYC- oder Ryzen 9-Prozessoren, die ein Multi-Chip-Module (MCM) Design mit mehreren Core Complex Dies (CCDs) verwenden, basiert die "Phoenix"-Architektur des 8700GE auf einem monolithischen Die.[8] Dieses Design bietet für den Hochfrequenzhandel einen signifikanten architektonischen Vorteil. Bei Multi-CCD-Designs erzwingt die Kommunikation zwischen Threads, die auf unterschiedlichen Dies lokalisiert sind, die Nutzung der Infinity Fabric. Diese Überquerung der Die-Grenzen induziert unweigerlich Latenzstrafen im Bereich von 60 bis 70 Nanosekunden.[10]

Der Ryzen 7 PRO 8700GE hingegen verfügt über einen einzelnen Core Complex (CCX), in dem alle acht Kerne untergebracht sind. Diese Kerne teilen sich einen unifizierten L3-Cache mit einer Kapazität von 16 Megabyte.[7] Durch die Restriktion der Ausführung auf diesen einzelnen CCX finden alle Inter-Thread-Kommunikationen, Cache-Kohärenz-Aktualisierungen und gemeinsamen Speicherzugriffe mit der absolut minimal physikalisch möglichen Latenz statt. Für einen Trading-Algorithmus, der darauf trainiert ist, >3% Short-Signale aus eintreffenden UDP-Shreds zu berechnen und unmittelbar eine signierte Transaktion an die Block Engine zu senden, ist diese deterministische Cache-Latenz von entscheidender Bedeutung.

Eliminierung von Simultaneous Multithreading (SMT)

Um das volle Potenzial der Architektur auszuschöpfen, ist ein tiefer Eingriff in die Basic Input/Output System (BIOS) Konfiguration zwingend erforderlich: Die Deaktivierung von Simultaneous Multithreading (SMT).[1] SMT erlaubt es dem Betriebssystem, zwei logische Threads auf einem physischen Kern auszuführen, um die Auslastung der Ausführungseinheiten

bei gemischten Workloads zu maximieren. Im Kontext von HFT führt SMT jedoch zu fatalem Nicht-Determinismus.

Zwei logische Threads, die denselben physischen Kern teilen, konkurrieren um essenzielle Hardwareressourcen: die L1-Daten- und Instruktions-Caches (jeweils 32 KB pro Kern beim 8700GE), den L2-Cache (1 MB pro Kern) sowie die Translation Lookaside Buffers (TLBs).[13] Wenn das Betriebssystem einen Hintergrundprozess auf dem logischen Sibling-Thread des primären Trading-Threads ausführt, kommt es zwangsläufig zur Verdrängung (Eviction) von hochrelevanten Marktdaten aus dem L1-Cache. In dem Moment, in dem die Preis-Prädiktions-Matrix auf diese Daten zugreifen muss, entsteht ein Cache-Miss, der einen teuren Zugriff auf den L3-Cache oder gar den Hauptspeicher erzwingt. Durch die vollständige Deaktivierung von SMT im BIOS wird der Prozessor auf acht reine, physische Kerne reduziert. Dem Trading-Algorithmus wird somit der exklusive und ungestörte Besitz seiner L1- und L2-Caches garantiert.

Kernel-Isolierung und das Tickless Operating System

Die Hardware-Ebene ist nur eine Komponente des Determinismus; das Linux-Betriebssystem selbst ist eine primäre Quelle für Jitter. Der Linux-Scheduler unterbricht kontinuierlich laufende Prozesse, um Hardware-Interrupts zu verarbeiten, Timer-Callbacks auszuführen oder CPU-Zeit auf andere Prozesse zu verteilen (Context Switching). Für einen Trading-Loop, der in einer Endlosschleife (Spin-Polling) einen Lockless-Ringbuffer auf eintreffende Jito-Shreds abfragt, stellt jeder vom Kernel induzierte Kontextwechsel eine Latenzspitze von mehreren Mikrosekunden dar – genug Zeit, um eine Arbitrage-Gelegenheit oder den optimalen Einstiegspunkt für den anvisierten >3% Short an einen Wettbewerber zu verlieren.

Die Lösung besteht in einer rigiden CPU-Isolierung, die dem Linux-Kernel beim Boot-Vorgang über spezifische Kernel-Parameter übergeben wird. Durch das Hinzufügen der Direktive isolcpus=2-7 in der GRUB-Konfiguration wird dem Linux-Scheduler explizit untersagt, allgemeine User-Space-Aufgaben auf den Kernen 2 bis 7 auszuführen. Die Kerne 0 und 1 werden dem System als "Housekeeping-Kerne" überlassen; sie verarbeiten das Betriebssystem, SSH-Sitzungen, Protokollierungsaufgaben und standardmäßige asynchrone Interrupts.

Zusätzlich muss ein "Tickless OS"-Zustand (Full Dynticks) etabliert werden. Standardmäßig nutzt der Linux-Kernel einen periodischen Timer-Interrupt (den Scheduling Tick, z.B. 1000 Hz), um Prozesse zu unterbrechen und die CPU-Zeitverwaltung zu aktualisieren. Durch die Parameter nohz_full=2-7 und rcu_nocbs=2-7 wird dieser Timer-Tick auf den isolierten Kernen vollständig abgeschaltet, vorausgesetzt, es läuft exakt ein einziger ausführbarer Thread auf dem Kern.[14] Der Parameter rcu_nocbs lagert zudem die Verarbeitung von Read-Copy-Update (RCU) Callbacks – eine asynchrone Speicherbereinigungstechnik des Kernels – auf die Housekeeping-Kerne (0-1) aus. Im Ergebnis erhalten die Trading-Threads auf den Kernen 2 bis 7 eine ununterbrochene, jitter-freie Ausführungsumgebung, die nahezu an die Eigenschaften

eines Real-Time Operating Systems (RTOS) heranreicht.

Speicherverwaltung und TLB-Miss Eliminierung durch 1GB Hugepages

Der Hetzner AX42-U Server stellt 64 GB DDR5-Arbeitsspeicher zur Verfügung, der zudem Error-Correcting Code (ECC) unterstützt.[2] DDR5 bietet eine immense Speicherbandbreite, die für das schnelle Einlesen von Netzwerkpaketen unerlässlich ist. Dennoch bleibt die Speicherlatenz ein kritischer Flaschenhals, insbesondere wenn die Übersetzung von virtuellen in physische Speicheradressen fehlschlägt.

Die Memory Management Unit (MMU) des AMD Ryzen-Prozessors nutzt den Translation Lookaside Buffer (TLB), um diese Adressübersetzungen zu cachen. Standardmäßig verwendet Linux eine Seitengröße (Page Size) von 4 Kilobyte (KB). Eine prädiktive Trading-Anwendung, die massive interne Zustandsmatrizen im Arbeitsspeicher vorhält – beispielsweise das kontinuierlich aktualisierte Orderbuch von Phoenix, den Zustand dutzender Whirlpool-Liquidity-Pools von Orca sowie die Gewichte des Machine-Learning-Modells zur Vorhersage der Short-Bewegungen – übersteigt schnell die begrenzte Kapazität des TLB.

Sobald der TLB erschöpft ist, kommt es zu einem TLB-Miss. Die MMU muss nun einen Page Table Walk durchführen: Eine hierarchische Suche durch die Seitentabellen im Hauptspeicher, die mehrere sequentielle, hochlatente DRAM-Zugriffe erfordert, bevor der eigentliche Datenzugriff überhaupt stattfinden kann.

Um diesen verheerenden Leistungsabfall zu eliminieren, wird die Architektur auf die Verwendung von 1 Gigabyte (1 GB) Hugepages umgestellt. Durch die Integration der Kernel-Parameter default_hugepagesz=1G hugepagesz=1G hugepages=32 werden beim Systemstart 32 Gigabyte des physischen Speichers exklusiv in Form von 32 zusammenhängenden 1-GB-Blöcken reserviert.[16] Ein einzelner TLB-Eintrag kann somit ein gesamtes Gigabyte an Arbeitsspeicher abbilden. Die Datenstrukturen der Trading-Anwendung sowie die Shared-Memory-Bereiche für die Netzwerkverarbeitung (AF_XDP UMEM) werden explizit in diesen Hugepages allokiert. Dies reduziert die Wahrscheinlichkeit von TLB-Misses für den kritischen Ausführungspfad auf nahezu null und gewährleistet deterministische Speicherzugriffszeiten im Nanosekundenbereich.

Die Unterstützung von ECC (Error-Correcting Code) durch den Speicherkontroller bietet zudem Schutz vor Single-Bit-Errors (SBEs), die durch kosmische Strahlung oder elektromagnetische Interferenzen verursacht werden können.[1] Während ECC-Speicher marginal höhere Basislatenzen aufweist als Non-ECC-Speicher, ist diese Stabilität für die Aufrechterhaltung der Systemintegrität während volatiler Marktphasen unverzichtbar. Ein Speicherfehler in den algorithmischen Gewichten des Prädiktionsmodells könnte andernfalls zu desaströsen Fehltrades führen.

Netzwerk-Tuning auf Layer 1 und 2: Interface und Ringbuffer

Die physische Netzwerkschnittstelle des AX42-U wird in der Regel von einem Intel I210 Gigabit Network Controller (oder äquivalenten Modellen wie I225/I226) angetrieben.[17] Ein Durchsatz von 1 Gbit/s ist für die Übertragung von Solana-Shreds und Transaktionen mehr als ausreichend; die kritische Metrik ist jedoch nicht die Bandbreite, sondern die absolute minimale Latenz vom Eintreffen des Photons im Glasfaserkabel bis zur Bereitstellung der Daten im L3-Cache des Prozessors.

Interrupt Coalescing und ethtool-Optimierung

Die Standardkonfiguration moderner Linux-Netzwerktreiber zielt auf Durchsatz und CPU-Effizienz ab. Eine zentrale Technik hierfür ist das Interrupt Coalescing (Interrupt Moderation). Anstatt für jedes eintreffende Netzwerkpaket einen Hardware-Interrupt auszulösen, wartet der Treiber eine vordefinierte Zeitspanne (oft 49 Mikrosekunden oder mehr) oder auf eine bestimmte Anzahl von Paketen, um diese dann gebündelt mit einem einzigen CPU-Interrupt zu verarbeiten.[18]

Für einen Handelsalgorithmus, der auf das Eintreffen eines spezifischen Jito-Shreds wartet, der eine massive Marktlücken-Verschiebung signalisiert, ist eine treiberseitige Verzögerung von 50 Mikrosekunden inakzeptabel. Mittels des Werkzeugs ethtool muss das adaptive Interrupt Coalescing strengstens deaktiviert werden.

ethtool Parameter	Konfiguration	Funktion im HFT-Kontext
adaptive-rx	off	Deaktiviert die dynamische Anpassung der RX-Interrupt-Raten durch den Treiber.
adaptive-tx	off	Deaktiviert die dynamische Anpassung der TX-Interrupt-Raten durch den Treiber.
rx-usecs	0	Setzt die Verzögerung für eingehende Pakete auf null. Erzwingt einen sofortigen Interrupt.[20]

tx-usecs	0	Setzt die Verzögerung für ausgehende Pakete (Transaktionen) auf null.[21]

Der Befehl ethtool -C eth0 adaptive-rx off adaptive-tx off rx-usecs 0 tx-usecs 0 zwingt die Netzwerkkarte, den Prozessor unverzüglich zu benachrichtigen, sobald ein Paket in den Hardware-FIFO-Puffer gelangt.[20] Diese Maßnahme erhöht die CPU-Last drastisch (oft auf 100% auf dem Kern, der die Interrupts verarbeitet). Da diese Interrupts jedoch mittels IRQ-Affinity (z.B. über das Schreiben einer Bitmaske in /proc/irq/X/smp_affinity) auf die nicht-isolierten Housekeeping-Kerne (0 und 1) gepinnt werden, bleibt die Ausführung der isolierten Trading-Threads auf den Kernen 2 bis 7 hiervon völlig unberührt.[23]

Dimensionierung der Hardware-Ringbuffer

Die Solana-Blockchain generiert während Hochlastphasen (z.B. bei begehrten NFT-Mints oder Panikverkäufen) extreme Spikes in der Netzwerkauslastung. Die UDP-Shreds vom Jito Block Engine können in massiven Mikrobürsts eintreffen. Wenn der Empfangsringbuffer (RX Ring) der Netzwerkkarte zu klein dimensioniert ist, werden Pakete auf der Hardware-Ebene verworfen (rx_missed_errors oder rx_dropped), bevor die Software sie überhaupt verarbeiten kann.[21]

Um dies zu verhindern, müssen die Hardware-Ringbuffer der Intel I210 Netzwerkkarte auf ihr absolutes Maximum vergrößert werden. Der Befehl ethtool -G eth0 rx 4096 tx 4096 maximiert die Anzahl der Deskriptoren für eingehende und ausgehende Pakete.[26] Ein größerer Ringbuffer erhöht zwar den Speicherverbrauch minimal, verhindert aber den katastrophalen Verlust von Marktdaten während kurzzeitiger Überlastungen, ohne zusätzliche Latenz in der Software-Queue zu erzeugen.

Netzwerk Layer 3 und 4: Hetzner Stateless Firewall und Conntrack Evasion

Die Netzwerksicherheit ist ein kritischer Aspekt, doch die standardmäßigen Linux-Firewalling-Mechanismen stehen im direkten Widerspruch zu den Latenzanforderungen des Hochfrequenzhandels. Insbesondere das Modul nf_conntrack (Netfilter Connection Tracking) stellt einen massiven Flaschenhals dar.

nf_conntrack ist dafür verantwortlich, den Zustand jeder aktiven Netzwerkverbindung in einer komplexen In-Memory-Hash-Tabelle zu speichern. Dies ermöglicht stateful Firewalls (die wissen, dass ein eingehendes Paket zu einer zuvor ausgehenden Anfrage gehört). Bei hohen Paketraten (Packets per Second - PPS), wie sie durch den konstanten Strom von UDP-Shreds generiert werden, führt das Connection Tracking zu schweren CPU-Sperrkonflikten (Lock Contention), Cache-Pollution und Performance-Einbrüchen, die die maximale Verarbeitungsrate um über 60 Prozent reduzieren können.[27]

Die Architektur dieses Systems löst dieses Problem durch einen zweistufigen Ansatz: Die Auslagerung der Zugangskontrolle auf die Switch-Ebene von Hetzner und die vollständige Umgehung von nf_conntrack im lokalen Kernel.

Hetzner Stateless Firewall API

Hetzner bietet eine hardwarebasierte, zustandslose (stateless) Firewall, die direkt auf dem Switchport operiert, an den der AX42-U angeschlossen ist.[29] Die Nutzung dieser Firewall entlastet die CPU des Servers vollständig von der Inspektion unerwünschter Pakete. Die Konfiguration erfolgt programmatisch über die Hetzner Robot API via JSON-Payloads, die per POST-Request an den Endpunkt /firewall/{server-id} gesendet werden.[29]

Da die Hetzner-Firewall stateless ist, muss die Regellogik präzise sowohl für den Ingress- als auch für den Egress-Traffic definiert werden. Für den Empfang der Jito Shredstream UDP-Daten wird eine dedizierte eingehende Regel erstellt, die den Verkehr auf die spezifischen IP-Adressen der Jito Block Engines beschränkt.

Struktur der JSON API-Payload für Jito Shredstream Ingress:

JSON Parameter	Wert / Bedeutung im Kontext
ip_version	"ipv4" – Die Jito-Infrastruktur nutzt primär IPv4.[30]
name	"Allow_Jito_Shreds" – Identifikator der Regel.[30]
protocol	"udp" – Shreds werden latenzoptimiert via UDP übertragen.[30]
src_ip	"64.130.50.14/32" – IP der Jito Frankfurt Block Engine.[32]
dst_port	"20000" – Der Port, an den der lokale Proxy gebunden ist (SRC_BIND_PORT).[33]
action	"accept" – Paket wird zum Server durchgelassen.[30]

Da ausgehender Datenverkehr ebenfalls gefiltert werden sollte, um Angriffsvektoren (wie DDoS-Amplifikation) zu minimieren, erfordert die Stateless-Architektur zwingend eine Regel, die die Antworten auf etablierte TCP-Verbindungen zulässt (etwa für die Yellowstone gRPC-Verbindung oder SSH-Zugriff). Dies wird erreicht, indem eine Ausgangsregel definiert

wird, die den Ephemeral-Port-Bereich (32768-65535) mit dem TCP-Flag ack erlaubt.[29]

Conntrack-Evasion im lokalen Kernel (NOTRACK)

Obwohl die Hetzner-Firewall den unerwünschten Traffic auf Hardwareebene blockiert, würde der Linux-Kernel den erlaubten, hochfrequenten Jito-Traffic (auf UDP-Port 20000) standardmäßig dennoch durch die nf_conntrack-Maschine leiten. Um dies zu verhindern, muss eine spezielle Regel auf der untersten Ebene des Netfilter-Stacks – in der raw-Tabelle der PREROUTING-Chain – implementiert werden.[37]

Unter Verwendung von nftables (dem modernen Nachfolger von iptables) wird folgende Logik implementiert: nft add rule ip raw prerouting udp dport 20000 counter notrack.[37]

Diese Regel inspiziert den Header jedes eintreffenden UDP-Pakets auf Port 20000 und weist den Kernel mit der Aktion notrack an, die Zustandsverfolgung für dieses spezifische Paket sofort zu umgehen. Das Paket wird ohne Allokation in der Conntrack-Tabelle und ohne die damit verbundene Latenz direkt an den Netzwerk-Stack – oder in diesem optimierten Fall an den AF_XDP-Socket – weitergeleitet.

AF_XDP Zero-Copy Paketverarbeitung

Selbst mit der Eliminierung von Interrupt Coalescing und Conntrack benötigt der traditionelle Linux-Netzwerk-Stack (TCP/IP) signifikante Rechenressourcen. Jedes Paket, das den Kernel durchläuft, erfordert die Allokation einer sk_buff Datenstruktur und das teure Kopieren der Payload-Daten (Memory Copy) vom Kernel-Space in den User-Space, was zusätzliche Latenzen und CPU-Cache-Pollution verursacht.[14]

Um die absolute Minimalgrenze der Software-Latenz zu erreichen, implementiert die Architektur AF_XDP (Address Family eXpress Data Path). AF_XDP ist eine Erweiterung des eBPF (Extended Berkeley Packet Filter) Frameworks und ermöglicht es, den Kernel-Netzwerk-Stack fast vollständig zu umgehen.[16]

Mechanismen und Speicherarchitektur von AF_XDP

Der Kern von AF_XDP ist der sogenannte UMEM (User Memory) – ein vom User-Space (der Trading-Applikation) reservierter, zusammenhängender Speicherbereich.[16] Dieser UMEM wird, wie zuvor beschrieben, zur Vermeidung von TLB-Misses in 1GB Hugepages allokiert. Der UMEM wird in gleich große Chunks (typischerweise 2048 oder 4096 Byte) unterteilt, in welche die Paketdaten direkt geschrieben werden.[16]

Die Kommunikation zwischen dem Netzwerktreiber (Intel I210) und der User-Space-Applikation erfolgt über vier Lockless-Ringbuffer [16]:

1. **FILL Ring:** Die Applikation legt Adressen (Offsets) von freien UMEM-Chunks in diesen

Ring.

2. **RX Ring:** Wenn ein Paket ankommt, nutzt der Treiber Direct Memory Access (DMA), um die Rohdaten des UDP-Shreds direkt, ohne CPU-Beteiligung, in den physischen Speicher des bereitgestellten UMEM-Chunks zu schreiben (Zero-Copy).[14] Anschließend legt der Treiber die Adresse dieses nun gefüllten Chunks in den RX Ring.
3. **TX Ring:** Für ausgehende Pakete (die auszuführende Short-Transaktion) legt die Applikation die Adresse des Chunks in den TX Ring.
4. **COMPLETION Ring:** Der Treiber signalisiert hierüber, dass das Paket gesendet wurde und der Chunk wieder freigegeben ist.

Das eBPF-Programm (XDP), das an den Netzwerktreiber angehängt wird, ist extrem minimalistisch. Es parst lediglich den Ethernet- und UDP-Header. Erkennt es ein Paket auf Port 20000 (Jito Shreds), ruft es die Helper-Funktion bpf_redirect_map auf, um das Paket direkt in eine BPF_MAP_TYPE_XSKMAP umzuleiten, welche mit dem AF_XDP-Socket der Trading-Applikation verbunden ist.[16] Für die Trading-Applikation auf den isolierten Kernen bedeutet dies: Sie liest die Shred-Daten direkt aus dem Hauptspeicher, ohne einen einzigen System Call (Syscall) auszuführen und ohne dass die Daten jemals den User-Space-Kontext verlassen haben.

Python IPC und die Umgehung des Global Interpreter Lock (GIL)

Während der AF_XDP Zero-Copy-Datenpfad für maximale Performance zwingend in hardwarenahen Sprachen wie C oder Rust implementiert werden muss [43], werden komplexe prädiktive Handelsmodelle – insbesondere solche, die Machine-Learning-Heuristiken nutzen, um die Wahrscheinlichkeit eines >3% Drops zu evaluieren – überwiegend in Python entwickelt. Die Integration des rasend schnellen Rust/C-Backends mit der Python-Logik stellt eine massive architektonische Hürde dar, vorrangig wegen Pythons Global Interpreter Lock (GIL) und dem langsamen IPC-Overhead (Inter-Process Communication) standardmäßiger Pipes oder UNIX-Sockets.

Jeder traditionelle IPC-Mechanismus erfordert Systemaufrufe (Syscalls) zur Datenübergabe, was die Latenzvorteile von AF_XDP sofort zunichtemachen würde. Die Lösung ist die Implementierung eines Lockless Shared Memory Ringbuffers zwischen dem Rust-Backend und dem Python-Frontend.[44]

Die SPSC-Architektur (Single-Producer, Single-Consumer)

Das AF_XDP-Empfangsprogramm (Rust) läuft auf einem isolierten Kern und fungiert als **Producer**. Es verarbeitet die Rohdaten der Solana-Shreds aus dem UMEM, validiert kryptografische Signaturen und extrahiert relevante Transaktions-Metadaten in ein dicht gepacktes C-Struct. Dieses Struct wird in einen speichergemappten Speicherbereich (mmap)

geschrieben, der direkt in den Adressraum des Python-Prozesses eingebunden ist.[40]

Der Python-Prozess (das prädiktive Modell) läuft auf einem anderen isolierten Kern und fungiert als **Consumer**. Mithilfe des Python-Moduls ctypes oder cffi [40] pollt die Python-Schleife kontinuierlich den Head-Pointer dieses Shared-Memory-Ringbuffers. Da es sich um ein SPSC-Design handelt, sind keine teuren, blockierenden Mutexe oder atomaren Compare-and-Swap (CAS)-Operationen erforderlich.[43] Die Speicherintegrität wird ausschließlich durch Memory Barriers (Acquire/Release-Semantik) gewährleistet, die verhindern, dass der Prozessor oder der Compiler die Reihenfolge der Lese- und Schreiboperationen vertauscht.[43]

Auf diese Weise wird der Python GIL effektiv umgangen. Die rechenintensiven Dekodierungsaufgaben finden parallel in Rust statt. Der Python-Prozess wird niemals blockiert, wartet nicht auf I/O und erhält die strukturierten Marktdaten (z.B. den Zustrom von Liquidität in einen Raydium-Pool, der den bevorstehenden Preissturz ankündigt) mit einer IPC-Latenz im Nanosekundenbereich.

Solana Daten-Ingestion: Jito Shredstream und Failover-Strategien

Die Identifikation von >3% Short-Bewegungen erfordert Daten, die jünger sind als der aktuell finalisierte Solana-Block. Wer auf RPC-Polling (getSlot, getMultipleAccounts) angewiesen ist, hat die Marktchance bereits verpasst, wenn die Daten eintreffen. Die Architektur nutzt stattdessen den direkten Zugang zu den Leader-Nodes über den Jito Shredstream.

Turbine-Bypass via Shredstream

Standardmäßig verteilt der Solana-Leader Block-Fragmente (Shreds) in einem strukturierten Baum-Muster, der sogenannten Turbine. Jeder Hop in diesem Baum induziert Netzwerkverzögerung.[31] Die Jito Shredstream-Dienstleistung durchbricht diesen Baum, indem sie VIP-Akteuren erlaubt, Shreds direkt von der Block Engine zu empfangen, oft hunderte Millisekunden bevor sie den Rest des Netzwerks erreichen.[33] Shreds sind UDP-Pakete, die unsortierte Transaktionsdaten, die Slot-Nummer und den Index innerhalb des Slots enthalten.[31]

Der Jito Shredstream Proxy wird lokal auf dem Hetzner AX42-U als nativer Prozess (oder im Host-Networking Docker-Container, um NAT-Overhead zu vermeiden) ausgeführt.[33] Er authentifiziert sich bei der Jito Block Engine mittels eines gewhitelisteten Solana-Keypairs (AUTH_KEYPAIR) und hält den Stream durch konstante Heartbeats aufrecht.[33]

Regionale Konfiguration und BGP-Failover

Die Konfiguration des Proxys ist entscheidend für die Ausfallsicherheit und Latenz. Die BLOCK_ENGINE_URL wird primär auf die europäische Instanz in Frankfurt

(https://frankfurt.mainnet.block-engine.jito.wtf) konfiguriert.[32] Die Konfiguration des Parameters DESIRED_REGIONS=frankfurt,amsterdam etabliert einen robusten Failover-Mechanismus.[33]

Jito Block Engine Region	Relevante IP-Adressen (Auszug)	Rolle in der Architektur
Frankfurt (GER)	64.130.50.14, 198.13.137.137, 64.130.40.25 [33]	Primäre Datenquelle. Geringste Latenz dank direkter Nähe zu Hetzner FSN1/NBG1.
Amsterdam (NED)	74.118.140.240, 202.8.8.174, 64.130.42.228 [33]	Sekundäre Failover-Quelle. Einziger Twin-Hub in Europa.[47] Übernimmt sofort bei Routing-Anomalien in Frankfurt.

Sollte das BGP-Routing (Border Gateway Protocol) zur Frankfurter Infrastruktur kurzzeitig gestört sein (etwa durch einen Link-Ausfall oder DDoS-Mitigation bei Transit-Providern), switcht der Proxy transparent auf die Amsterdamer Infrastruktur um. Der Proxy leitet die empfangenen Shreds auf den Parameter DEST_IP_PORTS=127.0.0.1:20000 um [33], wo sie vom zuvor konfigurierten AF_XDP eBPF-Programm abgefangen und in den UMEM geschrieben werden.

Yellowstone gRPC Optimierung für Account-States

Während Shredstream rohe Transaktionsdaten liefert, erfordern tiefgreifende Analysen des Systemzustands (z.B. der exakte Token-Kontostand eines Smart Contracts nach der Ausführung einer bestimmten Transaktion) verifizierte Account-Updates. Hierfür kommt das Yellowstone gRPC-Protokoll (Geyser) zum Einsatz, welches Zustandsänderungen direkt aus dem Speicher des Validators streamt.[48]

Das gRPC-Protokoll basiert auf HTTP/2. Um die Latenz des Streams von einem Remote-RPC-Provider (wie dRPC oder Helius) zum Hetzner AX42-U zu minimieren, muss das Protokoll tiefgreifend angepasst werden. Standardmäßig nutzen gRPC-Streams Kompressionsalgorithmen wie gzip oder zstd, um Bandbreite zu sparen. Das Entpacken der Payloads kostet jedoch wertvolle CPU-Zyklen und induziert inkonsistente Verzögerungen (Jitter).[50] In der Konfiguration des gRPC-Clients muss daher die Kompression strikt deaktiviert werden: "compression": {"accept":, "send":}.[50]

Darüber hinaus kappen Cloud-Load-Balancer und Firewalls inaktive TCP-Sitzungen oft nach 30 bis 120 Sekunden.[50] Um sicherzustellen, dass die HTTP/2-Verbindung zum

Yellowstone-Provider permanent "heiß" (warm) bleibt und Daten ohne Handshake-Verzögerung sofort fließen können, muss der Keepalive-Mechanismus angepasst werden. Durch Setzen der Parameter server_http2_keepalive_interval: "30s" und server_http2_keepalive_timeout: "10s" [50] sendet das System kontinuierlich Ping-Frames, was die TCP-Sitzung über Stunden hinweg aufrechterhält und die Latenz stabil im untersten Millisekundenbereich verankert.

BGP-Peering-Analyse und DE-CIX Frankfurt Routing

Die überragende Leistungsfähigkeit des Setups beruht maßgeblich auf der physikalischen Netzwerktopologie zwischen Hetzner und der Jito-Infrastruktur. Die Hetzner Data Center Parks in Falkenstein (FSN1) und Nürnberg (NBG1) sind über redundante Dark-Fiber-Verbindungen mit einer Kapazität von mindestens $n \times 100$ Gbit/s direkt mit dem Core-Backbone in Frankfurt verbunden.[5]

Frankfurt beheimatet den DE-CIX (Deutscher Commercial Internet Exchange), den weltweit führenden Internetknotenpunkt.[4] Die Latenz zwischen zwei Netzwerken wird primär durch die Anzahl der autonomen Systeme (AS) bestimmt, die ein Datenpaket durchqueren muss (BGP AS Hops). Jeder Hop über einen globalen Transit-Provider (wie Cogent, Telia oder Level3) fügt potenziell Warteschlangenverzögerungen und Routing-Overhead hinzu.

Hetzner nutzt aktives Peering am DE-CIX. Dies bedeutet, dass Datenverkehr, der für Jito-Server bestimmt ist, das Hetzner-Netzwerk verlässt und direkt auf der Switching-Fabric des DE-CIX an den Provider der Jito Block Engine (häufig Teraswitch oder Cherry Servers, AS-Nummern in den 30000er bis 60000er Bereichen) übergeben wird.[5] Diese direkten Private Network Interconnects (PNIs) oder öffentlichen Peering-Routen umgehen das öffentliche, unregulierte Internet weitestgehend.[6]

Dieses optimale BGP-Routing reduziert die "Round-Trip Delay" (RTD) auf ein physikalisches Minimum, limitiert beinahe ausschließlich durch die Lichtgeschwindigkeit im Glasfaserkabel (ca. 5 Mikrosekunden pro Kilometer).[54] Im Gegensatz zu Trading-Setups, die auf schwergewichtigen Cloud-Plattformen wie AWS (Amazon Web Services) oder GCP (Google Cloud Platform) gehostet werden – welche oft durch komplexe SDN-Overlays (Software Defined Networking) und virtuelle Hypervisoren ausgebremst werden –, bietet der Bare-Metal AX42-U in Kombination mit DE-CIX-Peering den entscheidenden Netzwerkvorteil, um Transaktionen für die >3% Short-Position mikrosekundenbruchteile früher in den Markt zu routen.[55]

Sub-Mikrosekunden Zeitsynchronisation: PTP und eBPF Fallback

Die exakte zeitliche Einordnung von On-Chain-Ereignissen ist für die Kalibrierung der prädiktiven Algorithmen essenziell. Um zu entscheiden, ob ein spezifischer UDP-Shred kausal mit einem Preisverfall auf einer Centralized Exchange (CEX, z.B. Binance) korreliert, müssen

die Zeitstempel im System mit absoluter Präzision arbeiten. Das traditionelle Network Time Protocol (NTP), realisiert durch Dämonen wie chronyd, operiert im User-Space und ist Netzwerk-Jitter sowie OS-Scheduling-Verzögerungen ausgesetzt. Die erreichbare Genauigkeit schwankt im Millisekundenbereich, was für HFT-Analysen ungenügend ist.

Um eine Sub-Mikrosekunden-Genauigkeit zu erreichen, greift die Architektur auf das Precision Time Protocol (PTP, IEEE 1588) zurück. Der verbaute Intel I210 Netzwerkcontroller besitzt native Hardwareunterstützung für PTP-Zeitstempel. Diese Funktion wird im Linux-Kernel über die Flags SOF_TIMESTAMPING_RX_HARDWARE und SOF_TIMESTAMPING_TX_HARDWARE exponiert.[57] Bei dieser Technologie stempelt die Hardware der Netzwerkkarte das eingehende Ethernet-Frame auf der MAC-Ebene (Media Access Control) exakt in dem Moment, in dem die Bits vom Kupfer- oder Glasfasermedium dekodiert werden.

Die Software-Implementierung nutzt den PTP-Dämon ptp4l, der sich mit einem Grandmaster Clock-Server im Rechenzentrumsumfeld (oder über Peering) synchronisiert. ptp4l adjustiert jedoch primär die Physical Hardware Clock (PHC) der Netzwerkkarte selbst. Um das Betriebssystem mit dieser extrem präzisen Hardwareuhr zu synchronisieren, wird das Tool phc2sys ausgeführt (z.B. phc2sys -s CLOCK_REALTIME -c eth0 -O 0 --step_threshold=0.5). Dies zwingt die Systemuhr von Linux in den Nanosekunden-Takt der Netzwerkkarte.[18]

Ein komplexes Problem entsteht jedoch durch die Nutzung von AF_XDP. Da AF_XDP den herkömmlichen Linux-Socket-Pfad umgeht (Zero-Copy), verliert die Applikation den Zugriff auf die Standard-Control-Messages (cmsg), über die SOF_TIMESTAMPING_RX_HARDWARE normalerweise an den User-Space übergeben wird. Hier kommt ein eBPF-Fallback ins Spiel. Das XDP-Programm, das an die Netzwerkkarte angehängt ist und die Pakete in den UMEM leitet, kann die Metadaten des Pakets (xdp_md) lesen.[40] Der PTP-Zeitstempel der Hardware wird von eBPF aus den Deskriptoren des Intel-Treibers extrahiert und in einen spezifischen Metadaten-Block vor die Payload des UDP-Shreds geschrieben, bevor das Paket in den Lockless-Ringbuffer umgeleitet wird. Somit erhält die Python-Trading-Logik für jeden eintreffenden Shred einen nanosekundengenauen Hardware-Zeitstempel.

Asynchrones I/O und Persistenz via io_uring und O_DIRECT für DuckDB

Obwohl das System auf Latenzminimierung in der Ausführung ausgelegt ist, erfordert die kontinuierliche Optimierung der prädiktiven Machine-Learning-Modelle das Aufzeichnen (Logging) massiver Datenmengen. Jeder verarbeitete Shred, jede berechnete Orderbuch-Tiefe und jede ausgeführte Transaktion muss gespeichert werden. Traditionelle Dateioperationen (wie write oder das erzwingende fsync) sind blockierende Systemaufrufe. Führt ein Trading-Thread auf einem isolierten Kern einen solchen Syscall aus, blockiert er, bis die NVMe-SSD den Schreibvorgang bestätigt, was die Trading-Schleife katastrophal unterbricht.[59]

Die Lösung liegt in der Nutzung von DuckDB, einer hochleistungsfähigen analytischen In-Process-Datenbank, unter expliziter Konfiguration des Linux io_uring-Frameworks.[61] io_uring revolutioniert asynchrones I/O (AIO), indem es zwei Shared-Memory-Ringbuffer – eine Submission Queue (SQ) und eine Completion Queue (CQ) – zwischen dem User-Space und dem Kernel aufbaut.[59]

Anstatt einen Syscall auszuführen, erstellt die Trading-Logik asynchron einen Eintrag (SQE - Submission Queue Entry) in der Submission Queue, wenn Daten in DuckDB geloggt werden sollen. Ein Hintergrund-Kernel-Thread (SQPOLL) erkennt diesen Eintrag und reicht den I/O-Befehl an den NVMe-Treiber weiter.[63] Wenn der Speichervorgang abgeschlossen ist, hinterlegt der Kernel eine Bestätigung (CQE) in der Completion Queue. Der ausführende Trading-Thread wird niemals blockiert, vollzieht keinen Kontextwechsel und verbleibt permanent im deterministischen Polling-State.[63]

Vermeidung von Double-Buffering mit O_DIRECT

Standardmäßig leitet Linux alle I/O-Schreibvorgänge durch den Page Cache (den Zwischenspeicher des Betriebssystems). Dieses Verhalten verursacht ein sogenanntes "Double Buffering": Daten werden aus dem RAM des Programms in den Kernel-Speicher kopiert, bevor sie auf die Festplatte gespült werden, was CPU-Zyklen verbraucht und den DDR5-RAM verschmutzt.[65]

Um dies zu unterbinden, wird DuckDB gezwungen, die Speicherdateien mit dem Flag O_DIRECT zu öffnen.[59] O_DIRECT umgeht den Linux Page Cache vollständig. In Kombination mit io_uring erlaubt dies dem NVMe-Controller, Direct Memory Access (DMA) zu nutzen. Die Daten werden direkt aus dem Speicher der DuckDB-Applikation auf das Software-RAID 1 der beiden 512 GB Gen4 NVMe SSDs [2] übertragen, ohne dass die CPU des AMD 8700GE eingreifen muss.[65] Diese Architektur gewährleistet, dass selbst kontinuierliches Logging von Gigabytes an historischen Marktdaten die Latenz der prädiktiven Short-Algorithmen in keiner Weise beeinträchtigt.

Fazit

Die Optimierung eines Hetzner AX42-U Bare-Metal-Servers für die Prädiktion und Ausführung von >3% Short-Positionen auf der Solana-Blockchain demonstriert eine kompromisslose Symbiose aus Hardware-Exzision und tiefgreifendem Kernel-Tuning. Durch die Ausnutzung der monolithischen Zen-4-Topologie des AMD Ryzen 7 PRO 8700GE und die Isolation der Rechenkerne bei gleichzeitiger Deaktivierung von SMT und OS-Scheduling-Ticks wird eine vollkommen deterministische Cache-Hierarchie geschaffen.

Die Identifikation von Marktanomalien geschieht verzögerungsfrei durch den Einsatz des Jito Shredstream-Proxys und Yellowstone gRPC, wobei das optimale BGP-Peering am DE-CIX in Frankfurt physikalische Distanzen minimiert. Die Implementierung von AF_XDP

Zero-Copy-Datenpfaden unter Umgehung von nf_conntrack durch Stateless-Firewalling transformiert das Paket-Ingestion-Modell in eine Lockless-Architektur, welche die Python-GIL via SPSC-Ringbuffer effektiv umgeht. Abgerundet durch PTP-Hardware-Zeitstempel und asynchrones NVMe-Logging via io_uring und O_DIRECT, entsteht eine Infrastruktur, die dem Hochfrequenzhandel nicht nur gewachsen ist, sondern die theoretischen physikalischen und softwarebasierten Latenzgrenzen der Systemarchitektur vollständig ausreizt.

Referenzen

1. The new Dedicated Server AX42 - The ideal ratio of price, performance and power consumption - Hetzner, Zugriff am April 8, 2026, https://www.hetzner.com/news/new-ax42/
2. AX42-U - Find cheap dedicated servers with AMD & Intel CPUs, Zugriff am April 8, 2026, https://www.hetzner.com/dedicated-rootserver/ax42-u/configurator/
3. AX42-U - Hetzner, Zugriff am April 8, 2026, https://www.hetzner.com/dedicated-rootserver/ax42-u/
4. DE-CIX Frankfurt – connect to the world's leading IX, Zugriff am April 8, 2026, https://www.de-cix.net/en/locations/frankfurt
5. Data centers and connection - Hetzner Docs, Zugriff am April 8, 2026, https://docs.hetzner.com/general/infrastructure-and-availability/data-centers-and-connection/
6. Seamless peering from Amsterdam to Frankfurt - DE-CIX, Zugriff am April 8, 2026, https://www.de-cix.net/en/remote-peering
7. AMD Ryzen™ 7 PRO 8700GE, Zugriff am April 8, 2026, https://www.amd.com/en/products/processors/desktops/ryzen-pro/8000-series/amd-ryzen-7-pro-8700ge.html
8. AMD Ryzen 7 8700GE Specs | TechPowerUp CPU Database, Zugriff am April 8, 2026, https://www.techpowerup.com/cpu-specs/ryzen-7-8700ge.c3493
9. AMD Ryzen 7 PRO 8700GE - Geekbench, Zugriff am April 8, 2026, https://browser.geekbench.com/v6/cpu/7005684
10. AMD Ryzen 7 4700GE Memory Benchmarked: Extremely Low Latency Explains Tiny L3 Caches | TechPowerUp Forums, Zugriff am April 8, 2026, https://www.techpowerup.com/forums/threads/amd-ryzen-7-4700ge-memory-benchmarked-extremely-low-latency-explains-tiny-l3-caches.269223/
11. I Tested Every Windows and BIOS Optimization for FPS (AMD Ryzen Edition + Benchmarks), Zugriff am April 8, 2026, https://www.youtube.com/watch?v=Vp332dU5xOU
12. Ryzen CCX Latency? : r/Amd - Reddit, Zugriff am April 8, 2026, https://www.reddit.com/r/Amd/comments/bc01e6/ryzen_ccx_latency/
13. Zen 4 - Wikipedia, Zugriff am April 8, 2026, https://en.wikipedia.org/wiki/Zen_4
14. eBPF Offload Native Mode XDP on Intel® Ethernet Linux Driver — ECI documentation, Zugriff am April 8, 2026, https://eci.intel.com/docs/3.0/development/tsnrefsw/bpf-xdp.html
15. AX Server – Possible configurations and add-ons - Hetzner Docs, Zugriff am April 8, 2026, https://docs.hetzner.com/robot/dedicated-server/server-lines/ax-server/

16. AF_XDP — The Linux Kernel documentation, Zugriff am April 8, 2026, https://www.kernel.org/doc/html/v4.18/networking/af_xdp.html
17. AX-Line and EX-Line with iNIC : r/hetzner - Reddit, Zugriff am April 8, 2026, https://www.reddit.com/r/hetzner/comments/1plfmpn/axline_and_exline_with_inic/
18. Austin's Nerdy Things - Nerding out, one post at a time, Zugriff am April 8, 2026, https://austinsnerdythings.com/
19. Chapter 12. Tuning interrupt coalescence settings | Network troubleshooting and performance tuning | Red Hat Enterprise Linux | 10, Zugriff am April 8, 2026, https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/network_troubleshooting_and_performance_tuning/tuning-interrupt-coalescence-settings
20. Intel Ethernet Adapters and Devices User Guide - Dell, Zugriff am April 8, 2026, https://dl.dell.com/manuals/all-products/esuprt_data_center_infra_int/esuprt_data_center_infra_network_adapters/intel-pro-adapters_user's-guide_en-us.pdf
21. Intel Ethernet Adapters and Devices User Guide - Dell, Zugriff am April 8, 2026, https://dl.dell.com/manuals/all-products/esuprt_data_center_infra_int/esuprt_data_center_infra_network_adapters/intel-pro-adapters_users-guide_en-us.pdf
22. How to achieve low latency with 10Gbps Ethernet - The Cloudflare Blog, Zugriff am April 8, 2026, https://blog.cloudflare.com/how-to-achieve-low-latency/
23. IRQ Affinity - 1.3 - ID:334019 | Intel® Ethernet 700 Series, Zugriff am April 8, 2026, https://edc.intel.com/content/www/us/en/design/products/ethernet/appnote-perf-tuning-guide-700-series-linux/%E2%80%8Birq-affinity/
24. How to Configure IRQ Affinity for Network Interfaces on Multi-Core Systems - OneUptime, Zugriff am April 8, 2026, https://oneuptime.com/blog/post/2026-03-20-configure-irq-affinity-network/view
25. Determine and Set Interrupt Affinity for Queues - UG1523, Zugriff am April 8, 2026, https://docs.amd.com/r/en-US/ug1523-x3522-user/Determine-and-Set-Interrupt-Affinity-for-Queues
26. Network Configuration (Stereo ace) | Basler Product Documentation, Zugriff am April 8, 2026, https://docs.baslerweb.com/network-configuration-%28stereo-ace%29
27. Stateless / no connection tracking iptables firewall possible? Answer: not possible., Zugriff am April 8, 2026, https://forum.proxmox.com/threads/stateless-no-connection-tracking-iptables-firewall-possible-answer-not-possible.31309/
28. franckferman/Floodles: Modular DoS/DDoS testing toolkit for authorized security audits. 19 attack vectors across L3/L4/L7. Python + C + Rust + Go. - GitHub, Zugriff am April 8, 2026, https://github.com/franckferman/Floodles
29. Firewall - Hetzner Docs, Zugriff am April 8, 2026, https://docs.hetzner.com/robot/dedicated-server/firewall/
30. Preface – Robot Webservice - Hetzner Robot, Zugriff am April 8, 2026, https://robot.hetzner.com/doc/webservice/en.html#post-firewall-server-id
31. How to decode ShredStream data of Jito? - Solana Stack Exchange, Zugriff am

April 8, 2026, https://solana.stackexchange.com/questions/12458/how-to-decode-shredstream-data-of-jito
32. Low Latency Transaction Send — Jito Labs Documentation - High Performance Solana Infrastructure, Zugriff am April 8, 2026, https://docs.jito.wtf/lowlatencytxnsend/
33. Low Latency Block Updates (Shredstream) — Jito Labs Documentation - High Performance Solana Infrastructure, Zugriff am April 8, 2026, https://docs.jito.wtf/lowlatencytxnfeed/
34. shredstream | Skills Marketplace - LobeHub, Zugriff am April 8, 2026, https://lobehub.com/de/skills/neversight-learn-skills.dev-shredstream
35. I have a server in hetzner and firewall is not working as sould be. | The FreeBSD Forums, Zugriff am April 8, 2026, https://forums.freebsd.org/threads/i-have-a-server-in-hetzner-and-firewall-is-not-working-as-sould-be.78079/
36. Stateless hetzner Firewall rules - with WHM Server causing issue. - cPanel Support, Zugriff am April 8, 2026, https://support.cpanel.net/hc/en-us/community/posts/34246742699287-Stateless-hetzner-Firewall-rules-with-WHM-Server-causing-issue
37. Disable conntrack for certain traffic to optimize performance : r/openwrt - Reddit, Zugriff am April 8, 2026, https://www.reddit.com/r/openwrt/comments/1iw5du1/disable_conntrack_for_certain_traffic_to_optimize/
38. 精通-Linux-管理第二版-全 - 博客园, Zugriff am April 8, 2026, https://www.cnblogs.com/apachecn/p/18967733
39. Easily managing your Firewall using nftables - Hetzner Community, Zugriff am April 8, 2026, https://community.hetzner.com/tutorials/firewall-using-nftables/
40. Deep Packet Capture in Python: A Complete Guide to Sniffing Techniques | by Ahmed Sobhi Ali | Medium, Zugriff am April 8, 2026, https://medium.com/@ahmedsobhialii/deep-packet-capture-in-python-a-complete-guide-to-sniffing-techniques-54d650e403e1
41. 5. AF_XDP Poll Mode Driver - Documentation - DPDK, Zugriff am April 8, 2026, https://doc.dpdk.org/guides/nics/af_xdp.html
42. AF_XDP - eBPF Docs, Zugriff am April 8, 2026, https://docs.ebpf.io/linux/concepts/af_xdp/
43. [Tutorial] Implementing AF_XDP in Rust process Millions of packets per second - Reddit, Zugriff am April 8, 2026, https://www.reddit.com/r/rust/comments/1qvfwp1/tutorial_implementing_af_xdp_in_rust_process/
44. skabbass1/ring-buffer: Lockless shared memory ring buffer - GitHub, Zugriff am April 8, 2026, https://github.com/skabbass1/ring-buffer
45. Implementing a Circular Buffer in Python | by Arashtad - Medium, Zugriff am April 8, 2026, https://medium.com/@arashtad/implementing-a-circular-buffer-in-python-c74e4b2e6a0c

46. Shredstream gRPC | RPCFast, Zugriff am April 8, 2026, https://docs.rpcfast.com/solana-dedicated-nodes/jito-shredstream-grpc
47. Premium Ryzen VPS and UDP Dedicated Shredstream Ryzen Premium Restocked in Amsterdam Region for Solana — Featuring the Latest Ryzen 9950X | by kishi.sol | Medium, Zugriff am April 8, 2026, https://medium.com/@KishiTheMechanic/premium-ryzen-vps-and-udp-dedicated-shredstream-ryzen-premium-restocked-in-amsterdam-region-for-1b1b6782d2a5
48. Solana Geyser gRPC Streaming — Setup Guide | dRPC, Zugriff am April 8, 2026, https://drpc.org/docs/solana-yellowstone-geyser-grpc
49. Monitor Solana Liquidity Pools with Yellowstone Geyser gRPC (Go) | Quicknode Guides, Zugriff am April 8, 2026, https://www.quicknode.com/guides/solana-development/tooling/geyser/yellowstone-go
50. Performance Tuning - Yellowstone gRPC - Mintlify, Zugriff am April 8, 2026, https://www.mintlify.com/rpcpool/yellowstone-grpc/operations/performance
51. Configuration Schema - Yellowstone gRPC - Mintlify, Zugriff am April 8, 2026, https://mintlify.com/rpcpool/yellowstone-grpc/api/config-schema
52. Reduced latency and optimized connectivity: Why direct data exchange boosts economic growth - DE-CIX, Zugriff am April 8, 2026, https://www.de-cix.net/en/resources/articles/reduced-latency-and-optimized-connectivity-why-direct-data-exchange-boosts-economic-growth
53. Private Network Interconnect (PNI) with Teraswitch - Cherry Servers, Zugriff am April 8, 2026, https://www.cherryservers.com/blog/pni-with-teraswitch
54. Latency kills: 5 reasons interconnection is the missing link in AI infrastructure - DE-CIX, Zugriff am April 8, 2026, https://www.de-cix.net/en/resources/articles/latency-kills-5-reasons-interconnection-is-the-missing-link-in-ai-infrastructure
55. Co-location Strategies for Solana Trading in 2026 - RPC Fast, Zugriff am April 8, 2026, https://rpcfast.com/blog/solana-trading-co-location-low-latency
56. ERPC Completes Major Upgrade of Solana Shared Endpoints — Ping-Based Auto-Routing and Enhanced Security Across All Regions, with Node Expansion in AMS and FRA | ELSOUL LABO B.V., Zugriff am April 8, 2026, https://labo.elsoul.nl/en/news/2025/10/07/erpc-shared-endpoints-ping-routing-upgrade/
57. Time Synchronization with Linux Precision Time Protocol - free5GC, Zugriff am April 8, 2026, https://free5gc.org/blog/20250709/20250709/
58. No timestamping capacities with Intel I210 on Ubuntu 22.04, Zugriff am April 8, 2026, https://community.intel.com/t5/Ethernet-Products/No-timestamping-capacities-with-Intel-I210-on-Ubuntu-22-04/m-p/1524378
59. io_uring for High-Performance DBMSs: When and How to Use It - arXiv, Zugriff am April 8, 2026, https://arxiv.org/html/2512.04859v1
60. The article claims that, when they switched to io_uring, > throughput increased ... - Hacker News, Zugriff am April 8, 2026, https://news.ycombinator.com/item?id=44624623

61. Flexible I/O for Database Management Systems with xNVMe - VLDB Endowment, Zugriff am April 8, 2026, https://vldb.org/cidrdb/papers/2026/p6-houlborg.pdf
62. Configuration - DuckDB, Zugriff am April 8, 2026, https://duckdb.org/docs/current/clients/c/config
63. OpenAnolis Whitepaper: Using io_uring to Improve Database System Performance, Zugriff am April 8, 2026, https://www.alibabacloud.com/blog/openanolis-whitepaper-using-io-uring-to-improve-database-system-performance_600342
64. Engineering a Columnar Database in Rust: Lessons on io_uring, SIMD, and why I avoided Async/Await : r/programming - Reddit, Zugriff am April 8, 2026, https://www.reddit.com/r/programming/comments/1qfkijn/engineering_a_columnar_database_in_rust_lessons/
65. Top 7 Zero-Copy I/O Moves for Real Throughput | by Bhagya Rana | Medium, Zugriff am April 8, 2026, https://medium.com/@bhagyarana80/top-7-zero-copy-i-o-moves-for-real-throughput-7ee225587c55
66. A Three-Tier Buffer Manager with CXL-Attached Memory for In, Zugriff am April 8, 2026, https://hpi.de/oldsite/fileadmin/user_upload/fachgebiete/rabl/Theses/master_theses/MA_Arbeit_Riekenbrauck-Niklas.pdf
67. How io_uring improves database performance | by Gaurav Sarma | Mar, 2026 - Medium, Zugriff am April 8, 2026, https://gauravsarma1992.medium.com/how-io-uring-improves-database-performance-7556d0d56dd2